

Intelligent Vehicles Exercise 2 Report:

Dead Reckoning Estimation



Group 2, Binay Kumar Shah & Kabir Tamari
D Intelligent Vehicles
February 11, 2026

1 Odometry with a differential drive robot

1.1 The kinematic model with the uncertainty parameters for the forward direction and heading

Task 1: Calculate the changes in the robot's forward direction (Δd) and heading ($\Delta\theta$), i.e., the changes expressed in the robot coordinate system caused by the latest movement. Also, calculate the variances (covariances) of (Δd) and ($\Delta\theta$). What uncertainties do you assume?

- **Change in Forward Distance and Heading:** These are calculated by the given equation:

$$\begin{aligned}dD &= (dDr + dDl)/2 \\ dA &= (dDr - dDl)/WHEEL_BASE;\end{aligned}$$

- **Covariance Calculation:**

$$\begin{aligned}CV &= (SIGMAR^2 - SIGMA1^2)/(2*WHEEL_BASE); \\ Cu &= [(SIGMAR^2 + SIGMA1^2)/4 \quad CV; \quad CV \quad (SIGMAR^2 + SIGMA1^2)/(WHEEL_BASE^2)];\end{aligned}$$

- **Aumme Uncertainties:**

- We assume that the left and right wheel encoder errors are independent and have equal variance i.e., $SIGMAR = SIGMA1$. Under this assumption, the cross-covariance term between Δd and $\Delta\theta$ becomes zero.
- We also assumed that wheel encoder has an error of 0.5 mm for every 12 mm traveled.
- The robot starts with a known heading of 90° and zero initial position uncertainty

Task 2: Calculate the new state variables of the robot under the assumption that the robot moves according to a circular trajectory (at this point, you could skip the compensation term given in Wang 1988 [1]). At the same time, calculate the covariance matrix of these new positions. You can assume that the robot always starts at the origin with a heading of 90° , i.e., $(x_0, y_0, \theta_0) = (0, 0, 90\pi/180)$.

- The following code calculates the change in position and updates the state variables using a circular approximation (the "half-angle" method) without the compensation term

```
dX = dD*cos(A(kk - 1)+dA/2);
dY = dD*sin(A(kk - 1)+dA/2);

X(kk) = X(kk-1) + dX;
Y(kk) = Y(kk-1) + dY;
A(kk) = mod(A(kk-1) + dA, 2*pi);
```

- The covariance matrix calculation is shown in the code part below:

```
Axya = [1 0 -dD*sin(A(kk-1) + dA/2);
0 1 dD*cos(A(kk-1) + dA/2);
0 0 1];

Au = [cos(A(kk-1) + dA/2) -dD*0.5*sin(A(kk-1)+dA/2);
      sin(A(kk-1) + dA/2) dD*0.5*cos(A(kk-1)+dA/2);
      0 1];

CV = (SIGMAR^2 - SIGMA1^2)/(2*WHEEL\_BASE);
Cu = [(SIGMAR^2 + SIGMA1^2)/4 \quad CV; \quad CV \quad (SIGMAR^2 + SIGMA1^2)/(WHEEL\_BASE^2)];
```

- **Result:** Figure 1 shows the output of the robot trajectory with no compensation applied.

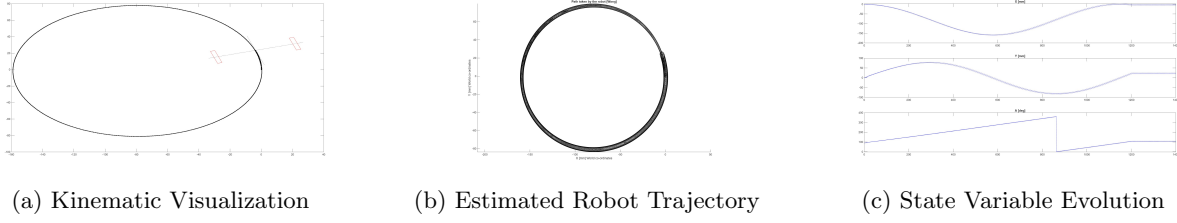


Figure 1: Robot path and state evolution with no compensation applied

Task 3: Run the entire sequence of the encoder, but also incorporate the compensation term given in the paper by Wang [1]. Are the estimated state variables and covariance matrices the same? If you read the encoder values less often, i.e., if you read them 2, 5, or maybe ten times as seldom or more, what will happen to the state variables? When do the state variables start to differ – and why? (Hint: Check, e.g., the Δd and $\Delta\theta$ values).

- **Comparison with Wang’s Compensation Term:** Figure 2 shows the output of the robot trajectory on applying compensation and it shows no visible differences from that of robot trajectory without compensation 1, which further proved by the figure ?? which is comparison graph of state and covariance plot with and without compensation. The difference between the model at the end of the path was found to be just **0.000051 mm**

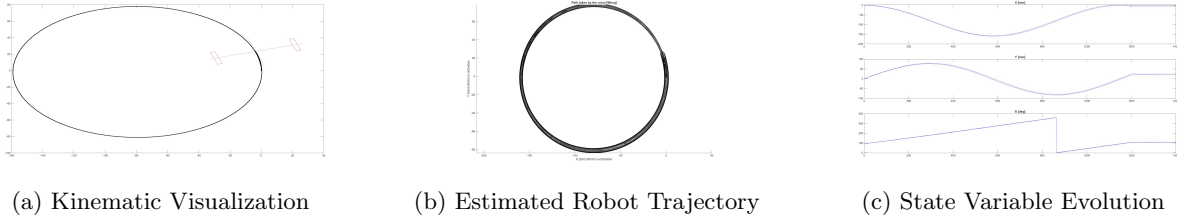


Figure 2: Robot path and state evolution with compensation applied

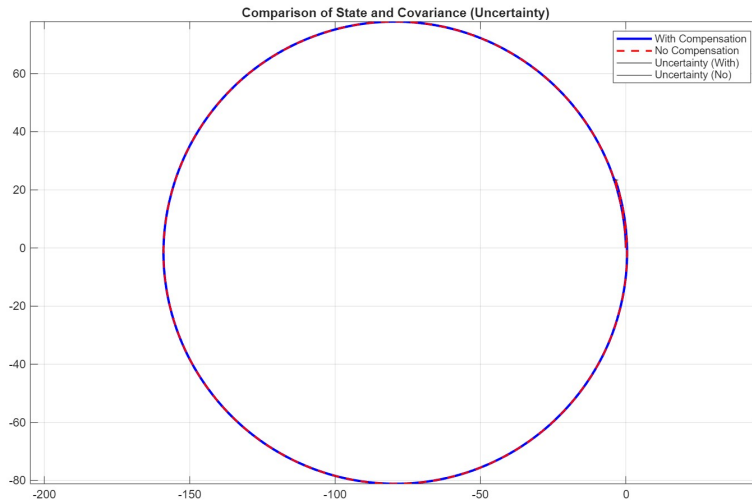


Figure 3: Comparison of State and Covariance (Uncertainty)

```

if COMP_TERM
    if abs(dA) > 1e-6 % Avoid division by zero
        factor = sin(dA/2) / (dA/2);
        dX = dX * factor;
        dY = dY * factor;
    end
end

% Code to compare the models
% Value of X, Y , A and P with no compsation:
X_no_comp = X;
Y_no_comp = Y;
A_no_comp = A;
P_no_comp = P;

% Value of X, Y , A and P with compsation:
X_with_comp = X;
Y_with_comp = Y;
A_with_comp = A;
P_with_comp = P;

>> figure;
% Plot Trajectories
plot(X_with_comp, Y_with_comp, 'b-', 'LineWidth', 2); hold on;
plot(X_no_comp, Y_no_comp, 'r--', 'LineWidth', 1.5);

% Plot the final uncertainty ellipses for both

plot_uncertainty([X_with_comp(end); Y_with_comp(end);
                  A_with_comp(end)], P_with_comp, 1, 2);
plot_uncertainty([X_no_comp(end); Y_no_comp(end);
                  A_no_comp(end)], P_no_comp, 1, 3);

legend('With Compensation', 'No Compensation',
'Uncertainty (With)', 'Uncertainty (No)');
title('Comparison of State and Covariance (Uncertainty)');
axis equal; grid on;

% Code to Calculate the Euclidean distance between the two final points
final_diff = sqrt((X_no_comp(end) - X_with_comp(end))^2
                  + (Y_no_comp(end) - Y_with_comp(end))^2);

fprintf('The difference between models at the end of
        the path is: %.6f mm\n', final_diff);

```

- **Effects of Reduced Sampling Rates:** Figure 4 shows the impact on state variables on changing sampling rate from 2, 5 to 10. From the comparison it is concluded that as the sample interval increases, the circular path begins to show chord approximations and for the angles updates are much sharper because each step covers more ground leading to a visible divergence in the state over time.
- **State variable differ:** From the heading comparison of figure 4 we can see that the state starts to differ right from the beginning. While the distance (δX) remain similar, the orientation ($\delta \theta$) begins to drift. This causes later X and Y variables to drift as the heading error gets integrated into the position.

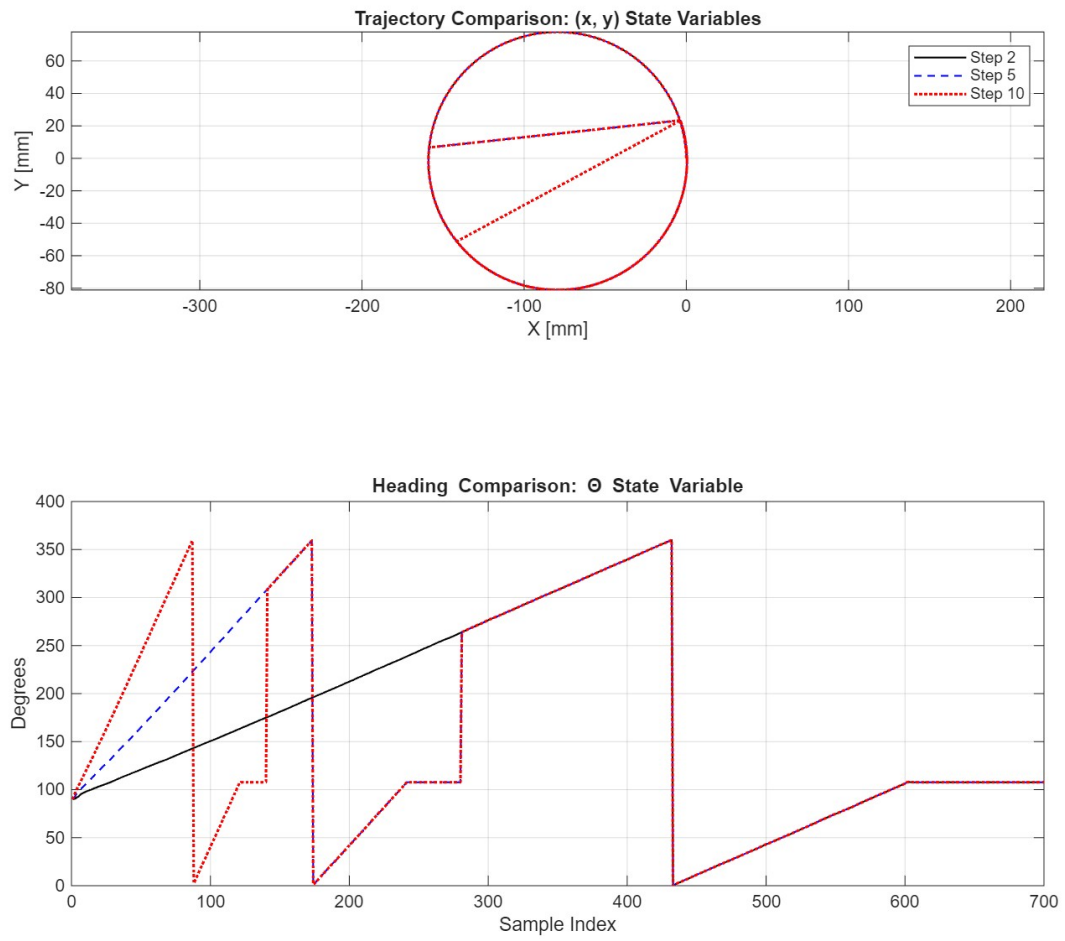


Figure 4: Trajectory and heading comparison at sampling rate 2, 5 and 10

Task 4: How are the covariance matrices evolving during the run (plot the entire run together with the calculated covariance matrices)? You don't have the actual values of the state variables (a common problem in the mobile robot community) – but are the uncertainties realistic, nonetheless?

In order to visualize the covariance matrices evolving more clearly, plotting was done after 50th step as shown by the code below and hence we got a plot as shown in figure 5. The result clearly shows that the ellipses get larger and more elongated as the robot moves during the run. This highlights that the model is realistic despite not having an actual value of state variable because in dead-reckoning, error are cumulative and once an error occurs, every subsequent calculations uncertainty grows as well.

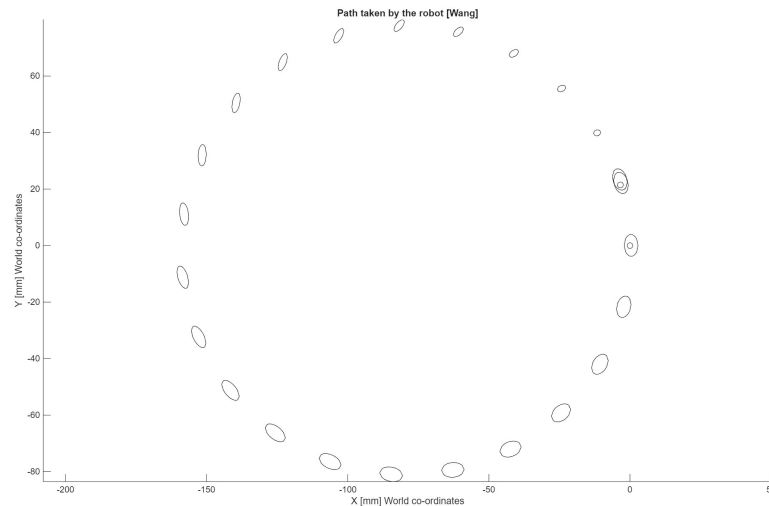


Figure 5: Evolving of Covariance matrices

```
....
hold on;
% plotting the covariance after 50th step
for kk = 1:50:N,
    C = [P(kk,1:3);P(kk,4:6);P(kk,7:9)];
    plot_uncertainty([X(kk) Y(kk) A(kk)]', C, 1, 2);
end;
hold off;
....
```

1.2 The kinematic model (different from Task 1-4) with the uncertainty parameters, distance driven by the left and right wheel

Task 5: Once more, run the same test but use less well-known wheel diameter and wheelbase values, e.g., $D = 14\text{mm}$ and $WB = 45\text{mm}$. How does it affect the estimated state variables? Plot the trajectories in the same plot as the other trajectories. What assumptions about these errors would you make?

We performed the dead-reckoning simulation with modified parameters: Wheel Diameter $D = 14\text{ mm}$ (vs. standard 15.3 mm) and Wheelbase $WB = 45\text{ mm}$ (vs. standard 53 mm). Then the estimated trajectory differs significantly from the standard one, as shown in the comparison plot 6.

1. **Effect of Reduced Wheel Diameter (D):** The incremental distance traveled Δs is proportional to D .

$$\Delta s \propto D$$

Using $D = 14\text{ mm}$ instead of 15.3 mm causes an underestimation of the distance traveled by a factor of $\frac{14}{15.3} \approx 0.915$. Consequently, the total length of the estimated path is approximately 8.5% shorter than the standard path.

2. **Effect of Reduced Wheelbase (WB):** The change in orientation $\Delta\theta$ is inversely proportional to WB .

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{WB}$$

Using $WB = 45\text{ mm}$ instead of 53 mm causes an overestimation of the turning angle by a factor of $\frac{53}{45} \approx 1.18$. This means the estimated robot orientation changes more rapidly than the standard estimation. For a circular path, this results in a tighter predicted turning radius, causing the trajectory to spiral inward or form a smaller circle.

Assumption: We can assume that the diameter error acts as a linear scaling factor on the path length, and the wheelbase error acts as a scaling factor on curvature. This is particularly critical because orientation errors integrate into position errors, causing the estimated path to diverge rapidly from the true path (accumulation of error).

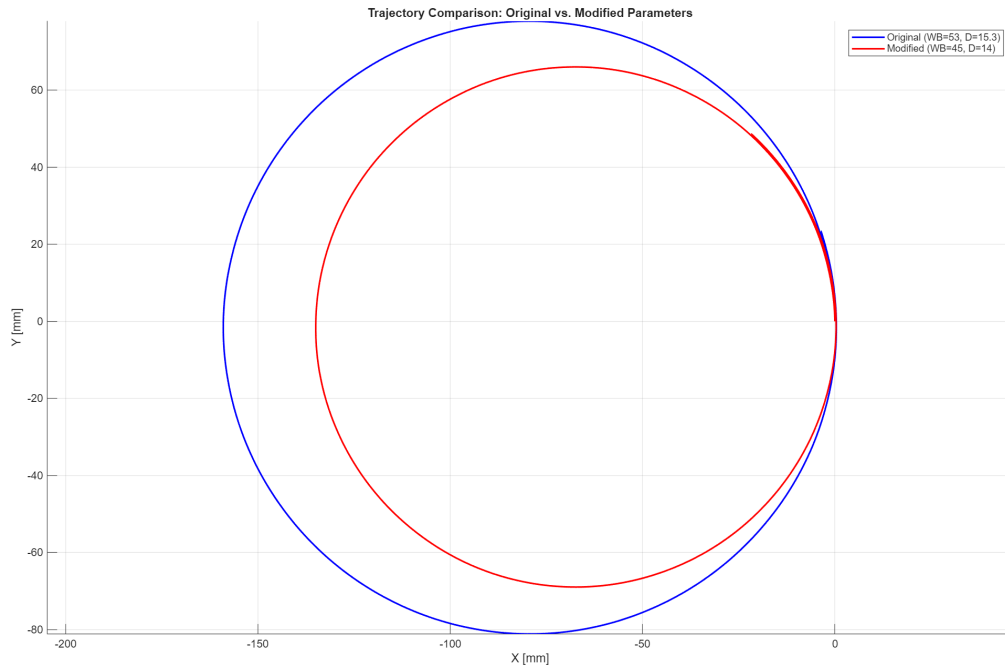


Figure 6: Standard Wheelbase vs Modified Wheel Base

Task 6: Repeat the experiment in task 5, but use the data file 'khepera.txt'. Plot the trajectories in the same plot as the other trajectories, i.e., different wheelbase and wheel diameter!

Figure 7 shows the comparison plotting for 'khepera.txt' file. We can observe that, the drift started shortly after the starting motion, where small errors in θ are the leading cause of position drift.

Moreover, we also can observed that the error are not random in that case it would have been wiggling around, rather found to be a systematic error because the red path over the time follows a consistently different pattern, it proves model is misinterpreting the physical reality of the robot's motion.

At last the error, the covariance matrices grows and elongated over time, demonstrating that the model in dead-reckoning.

This all highlight one important rules in odometry that the precise calibration is mandatory.

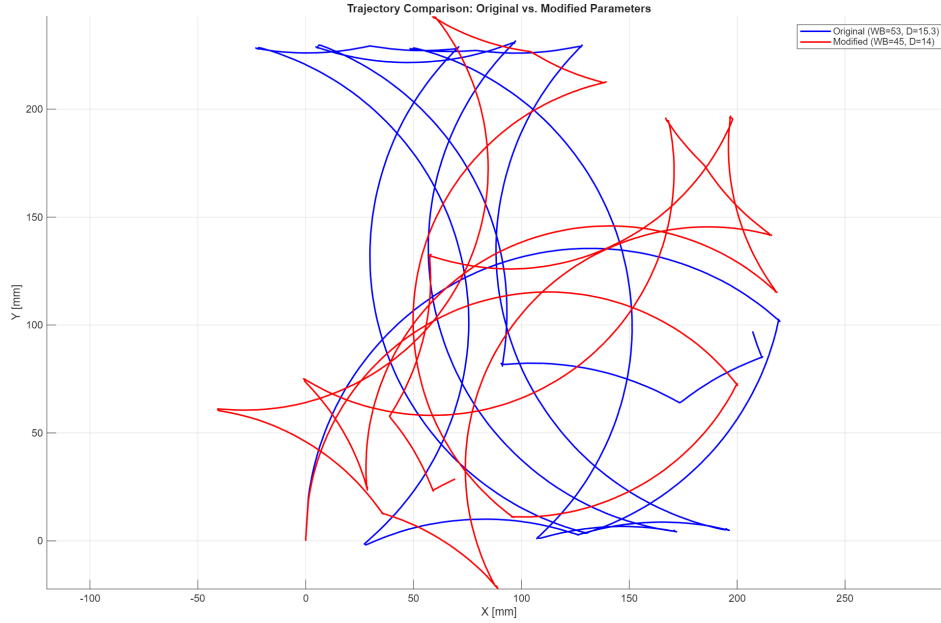


Figure 7: Trajectories comparison between original and modified parameter for khepera.txt

2 Dead-reckoning using a Three-Wheeled vehicle

Task 7: Calculate the new state variables of the robot. Also, calculate the covariance matrix of these new positions.

The robot's new state (x_k, y_k, θ_k) is calculated using a 2nd-order Runge-Kutta (midpoint) discrete kinematic model. The equations for the state update are:

$$x_k = x_{k-1} + v_k \Delta t \cos(\theta_{k-1} + \frac{v_k \Delta t}{2L} \sin(\alpha_k)) \quad (1)$$

$$y_k = y_{k-1} + v_k \Delta t \sin(\theta_{k-1} + \frac{v_k \Delta t}{2L} \sin(\alpha_k)) \quad (2)$$

$$\theta_k = \theta_{k-1} + \frac{v_k \Delta t}{L} \sin(\alpha_k) \quad (3)$$

where v_k is the speed, α_k is the steering angle, Δt is the sampling time, and L is the wheelbase. The covariance matrix of the new position, P_k , represents the uncertainty in the state which is calculated using law of error propagation given in the exercise pdf.

$$P_k = \nabla f_x P_{k-1} \nabla f_x^T + \nabla f_u Q_u \nabla f_u^T \quad (4)$$

Here:

- P_{k-1} is the covariance of the previous state.
- $Q_u = \text{diag}(\sigma_v^2, \sigma_\alpha^2)$ is the covariance matrix of the input process noise (speed and steering).
- ∇f_x is the Jacobian matrix of the motion model with respect to the state vector $[x, y, \theta]^T$.
- ∇f_u is the Jacobian matrix of the motion model with respect to the control inputs $[v, \alpha]^T$.

This equation propagates the existing uncertainty and adds the new uncertainty arising from the noisy control inputs. The following MATLAB code snippet implements the state update and covariance propagation for Task 7. It calculates the Jacobians ∇f_x ('Axya') and ∇f_u ('Au') and updates the covariance matrix 'P'.

```
% Change of relative movements
dD = v * cos(a) * T;
dA = (v * sin(a) * T) / L;

% Calculate the change in X and Y (World co-ordinates) using Midpoint
theta_mid = A(kk-1) + dA/2;
dX = dD * cos(theta_mid);
dY = dD * sin(theta_mid);

% Predict the new state variables
X(kk) = X(kk-1) + dX;
Y(kk) = Y(kk-1) + dY;
A(kk) = mod(A(kk-1) + dA, 2*pi);

% Jacobian matrix w.r.t. state (x, y, theta)
Axya = [1, 0, -dD * sin(theta_mid);
        0, 1,  dD * cos(theta_mid);
        0, 0,  1];

% Partial derivatives w.r.t inputs (v, alpha)
% derived from state update equations
df_dv = [T*cos(a)*cos(theta_mid) - (dD*T*sin(a)/(2*L))*sin(theta_mid);
        T*cos(a)*sin(theta_mid) + (dD*T*sin(a)/(2*L))*cos(theta_mid);
        (T*sin(a))/L];
```

```

df_da = [-v*T*sin(a)*cos(theta_mid) - (dD*v*T*cos(a)/(2*L))*sin(theta_mid);
         -v*T*sin(a)*sin(theta_mid) + (dD*v*T*cos(a)/(2*L))*cos(theta_mid);
         (v*T*cos(a))/L];

Au = [df_dv, df_da];

% Process noise covariance
Cu = [sigma_v^2, 0;
      0, sigma_a^2];
Cxya_new = Axya*Cxya_old*Axya' + Au*Cv*Au';

```

Task 8: Run the sequence and compare estimated state variables to ground truth. Also, compare the error in the state variable estimates to the standard deviations of the estimated variances. What errors do you assume in the steering angle and the speed?

For a bit of experiment and to understand the impact of our error assumption we assumed 2 sets of steering angle and speed:

- Scenario 1 (Low Noise): $\sigma_v = 5 \text{ mm/s}$, $\sigma_\alpha = 1^\circ$.
- Scenario 2 (High Noise): $\sigma_v = 15 \text{ mm/s}$, $\sigma_\alpha = 10^\circ$.

We ran the simulation 2 times with those variables and path estimated path taken by the robot (our state variables) and the actual ground truth is plotted below:

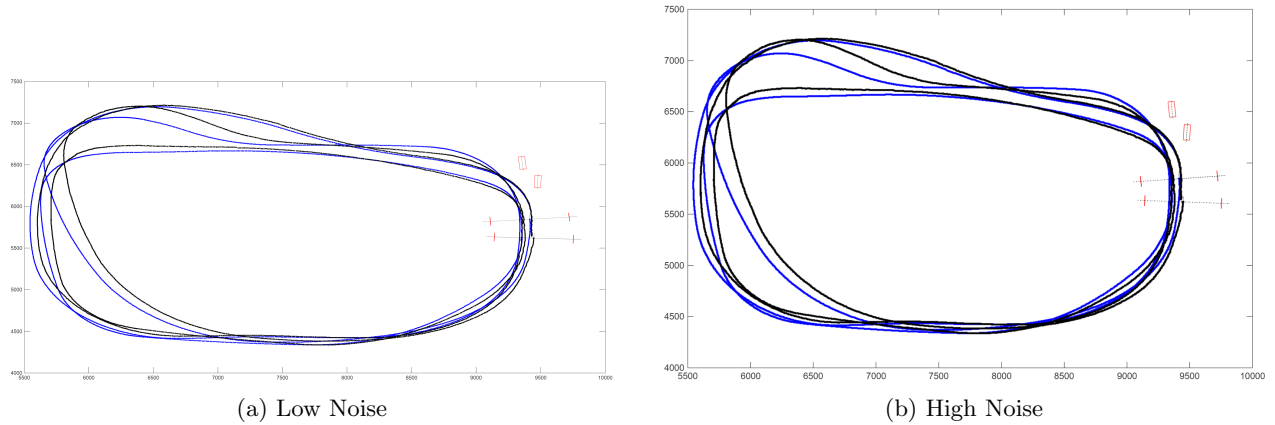


Figure 8: Estimated State vs Ground Truth.

The error ellipses represent the shape of the position uncertainty covariance matrix (P_k). As the robot moves, these ellipses grow and rotate. Uncertainty accumulates over time because dead reckoning is an integration process so errors in previous steps propagate to future steps. In Scenario 2, the ellipses are vastly larger (Figure 8b) because the steering angle noise ($\sigma_\alpha = 10^\circ$) has a massive impact on the lateral position uncertainty over time.

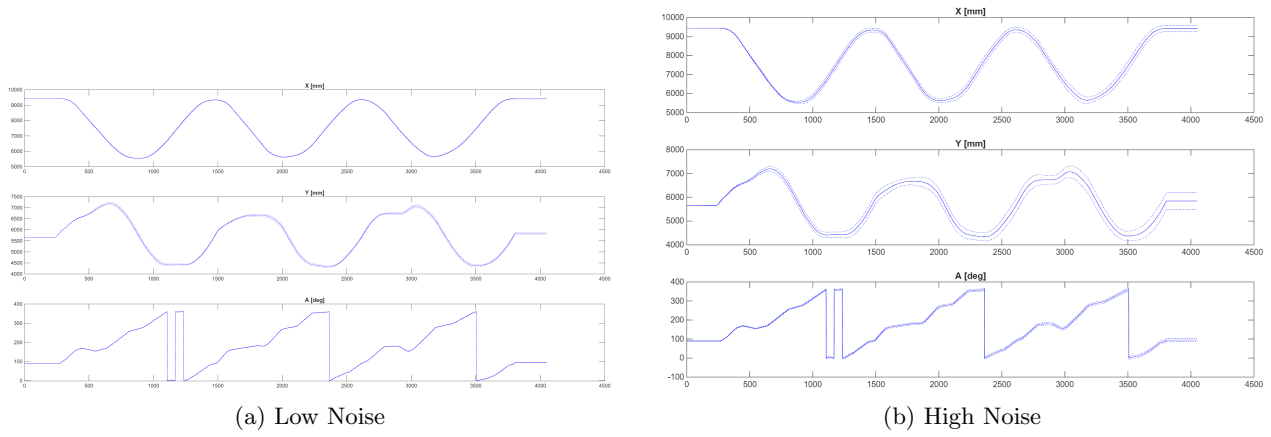


Figure 9: Error vs. Uncertainty

Figure 9 shows that our actual error (blue line) remains largely bounded by the estimated σ curves. In the high-noise case (b), the error magnitude is naturally larger, but the estimated σ also scales up to encompass it.

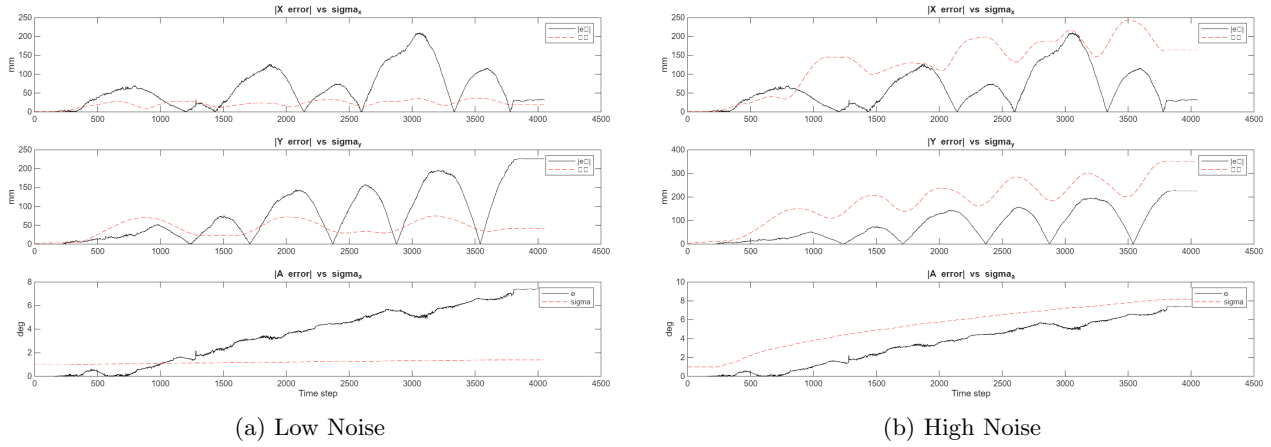


Figure 10: Absolute Error vs Standard Deviations of Estimated Variances

Figure 10 compares the absolute error against against the theoretical 1σ standard deviation boundaries derived from the covariance matrix P . The fact that the absolute error frequently exceeds the bounds indicates that the noise parameters (σ_v and σ_α) are currently under-modeled, meaning the robot is experiencing more mechanical noise than the simulation parameters suggest. The dips in position error ($|e_x|, |e_y|$) occur because the robot's erroneous path physically crosses the ground truth path ("Path Crossing"), momentarily reducing the distance error to zero. This is coincidental, not a correction. In contrast, the Heading Error grows consistently, confirming the robot is drifting. The humps reflect the robot oscillating around the true path while turning.

Task 9: Plot the error ellipses along the path. How long (distance and time) is the path?

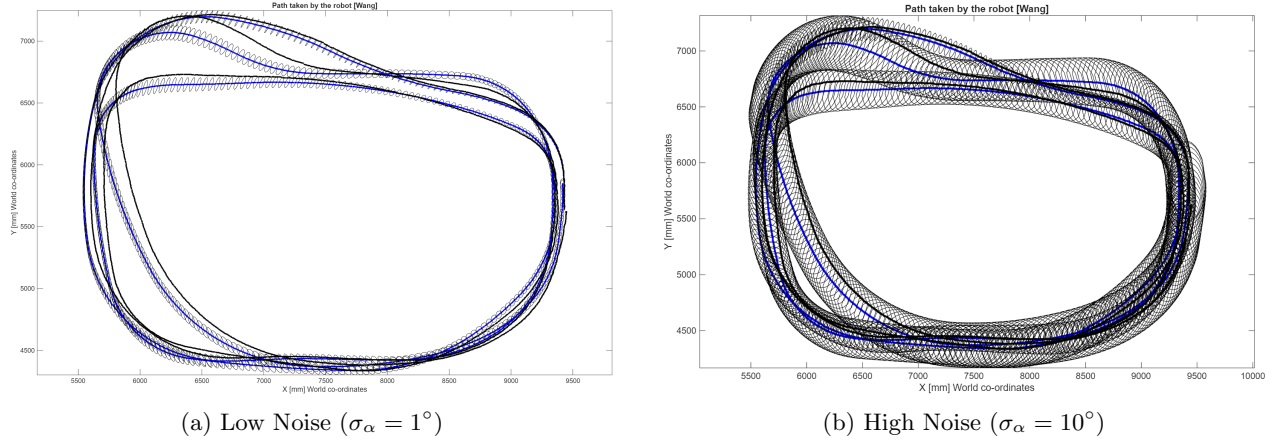


Figure 11: State variables vs Ground Truth with error ellipse bounds

In Scenario 1 (*LowNoise*), the estimated state tracking is good. The 1σ confidence bounds (dotted lines) are narrow, and the ground truth (solid/cross) stays well within them. This indicates that with low sensor noise, dead reckoning can provide a reliable pose estimate over short durations.

In Scenario 2 (*HighNoise*), the uncertainty grows much faster. The 1σ bounds in Figure 11(b) diverge significantly from the mean estimate. However, importantly, the ground truth still remains within these wider bounds. This confirms that our calculation is consistent as it correctly knows that it is uncertain. If the ground truth had drifted outside the σ bounds, it would indicate an issue with our noise modeling or Jacobian derivation.

Path Statistics: Calculated from the simulation data:

- **Total Time:** 202.55 seconds.
- **Total Distance:** 11.23 meters.

3 Conclusion

In this exercise, we successfully implemented a dead-reckoning estimation algorithm for a three-wheeled robot using a kinematic bicycle model. By propagating both the state and the covariance matrix through prediction step, we were able to estimate the robot's pose and quantify the uncertainty associated with its movement.

Practically, this implies that while dead reckoning provides a smooth short-term estimate, it is insufficient for long-term navigation. An autonomous system needs to fuse this relative estimate with absolute position measurements to collapse the uncertainty ellipses and correct the inevitable drift.